SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Pune – 412115, Maharashtra State, India
https://www.sitpune.edu.in/
DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING
AY 2025-26

Symbiosis Institute of Technology, Pune

B.Tech, Artificial Intelligence and Machine Learning

CA-3: Agentic AI Proof of Concept

PROJECT REPORT

Multi-Agent SOC Alert Triage and Incident Response Assistant

Submitted by

Dakshesh Chauhan- 24070126048

Bhavya Tanna- 24070126046

Subject faculty

Prof. Harshal Raut

Abstract

Six cooperating agents in a LangGraph state graph turn a raw batch of security alerts into ranked, evidence-backed incidents. They also produce a response playbook, a tested detection rule exported as Sigma, tickets and a report. A local LLM (Ollama, llama3.2 3B) plans each investigation and writes prose. Grouping, scoring, routing and every pass/fail decision are deterministic Python.

A Critic agent checks the work at two checkpoints and sends faulty work back to the agent that made it. It also fact-checks every sentence the LLM writes. An analyst approves containment through a LangGraph interrupt, and their verdicts are remembered for the next run.

On the 153-alert demo dataset the system finds all 4 hidden attack chains and escalates none of the 127 benign alerts (precision, recall and F1 all 1.0). Across 15 generated datasets it finds 57 of 60 chains. In an ablation, the Critic catches and corrects all 42 injected faults; with it disabled, all 42 reach the output.

Table of Contents

1. Introduction and problem statement 3
2. Objectives and scope 3
3. System architecture 4
4. The six agents and their tools 5
5. Orchestration: the LangGraph state graph 6
6. Severity scoring and decision logic 6
7. Validation: the Critic 7
8. Human in the loop and analyst memory 8
9. Implementation 9
10. Data and experimental setup 10
11. Results and evaluation 10
    11.1 Demo dataset (seed 42, 153 alerts) 11
    11.2 Benchmark across 15 datasets 11
    11.3 Critic ablation 12
    11.4 Runs with the real Ollama model 12
12. Demo walkthrough 12
13. Limitations and future work 13
14. Conclusion 14
15. References 14
Appendix: how to run 14

1. Introduction and problem statement

A Security Operations Centre (SOC) receives far more alerts than its analysts can investigate by hand. Most are benign: failed logins, patch installs, routine VPN sessions. The few that matter are spread across several alerts, hosts and minutes, so they look harmless one at a time. This is alert fatigue: real attacks get missed or are found late because analysts triage alert by alert.

Triage is a multi-step job. An analyst must group related alerts, look up indicators and assets, connect the steps of an attack, judge how serious it is, decide what to do, write a detection rule and hand over the case. A plain chatbot cannot do this. It cannot call tools, its answers are not checked, and it may invent indicators or hosts that were never in the data.

This project builds an agentic proof of concept for that workflow. Separate agents plan, investigate, correlate, validate, respond and report, and the orchestration between them is visible. The design rule throughout is that the language model may plan and write, but it never decides: every score and every pass/fail check is plain Python that can be tested and explained.

Domain: Cybersecurity Operations (SOC). **Course:** CA-3 Agentic AI Proof of Concept, SIT Pune.

2. Objectives and scope

The proof of concept had to show five agentic behaviours visibly, each mapped to a part of the system:

Behaviour required	Where it happens
Task planning	Planner builds a task plan of candidate incidents; Research lets the LLM plan which tools to call
Tool execution	Research calls threat intel, asset inventory, user baseline and estate-hunt tools; Software Development runs its rule tester and pySigma
Decision-making	Deterministic severity rubric and router: auto-close, analyst queue or escalate
Validation	Critic checkpoints 1 and 2, plus the fact-check of all LLM prose
Agent collaboration	Shared LangGraph state; the Critic sends work back to Research, the Executor or Software Development

Further objectives

- Run entirely free and local: Python, LangGraph and a local Ollama model, no paid API.
- Keep a human in the loop for containment, and remember analyst verdicts across runs.
- Measure the result against ground truth and show what the Critic adds with an ablation.

In scope: synthetic alerts, a Windows Security and Sysmon sample, mock integrations (threat intel, ticketing, notifications), a local web console.

Out of scope: real SIEM connections, real threat-intel feeds, real containment actions, authentication and deployment.

3. System architecture

Alerts flow through six agents inside one LangGraph state graph. Two Critic checkpoints can send work back, and an analyst approval step sits before any response action. The local LLM sits outside the graph: agents call it, and the Critic checks everything it returns.

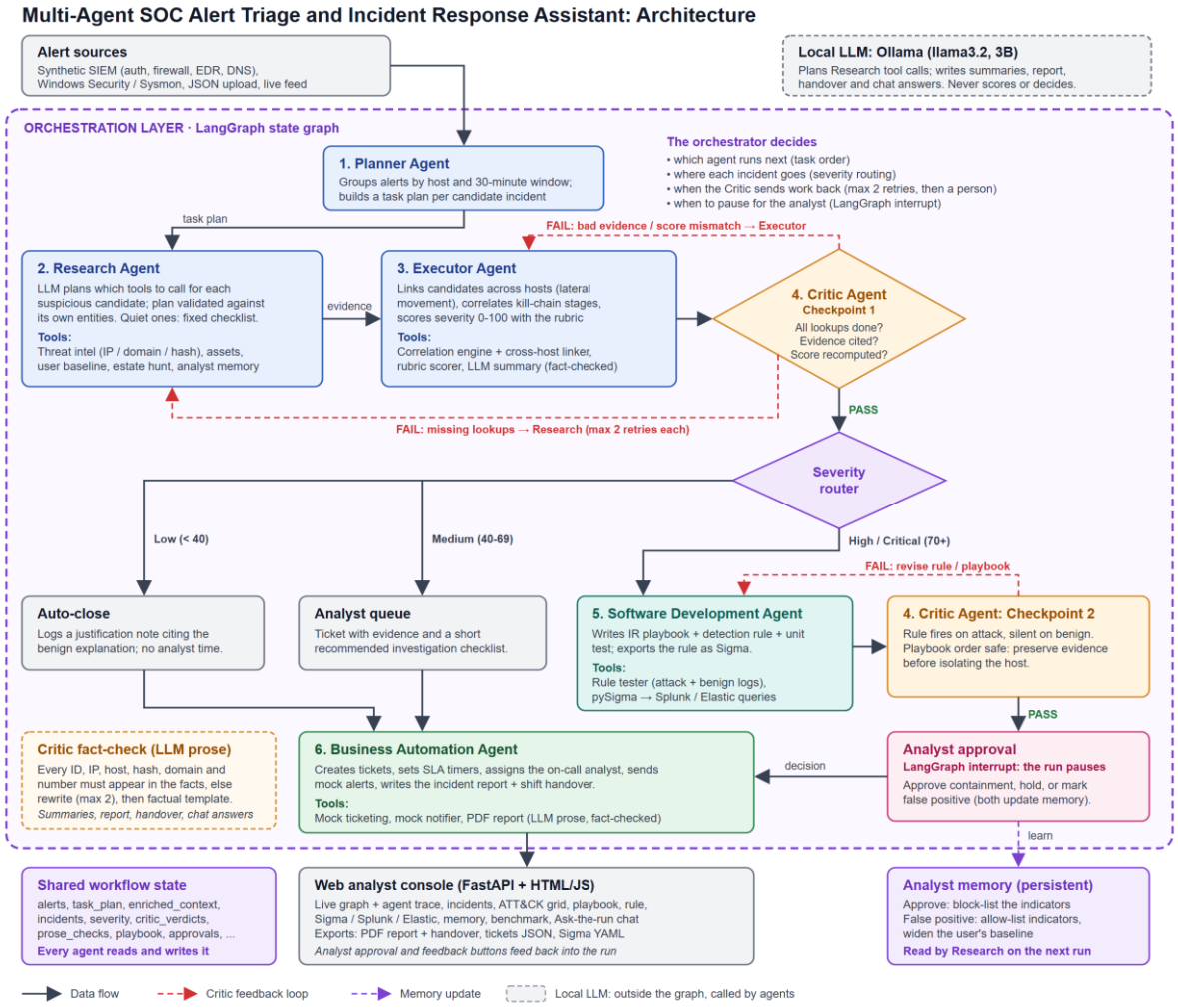


Figure 1. System architecture. Solid arrows are data flow, red dashed arrows are Critic feedback loops and the purple dashed arrow is a memory update.

The architecture has four layers:

- **Inputs:** synthetic SIEM alerts (authentication, firewall, EDR, DNS), a Windows Security and Sysmon sample, or an uploaded JSON file, optionally replayed as a live feed.
- **Orchestration:** the LangGraph graph with the six agents, the severity router and the analyst approval interrupt. All agents read and write one shared state object (alerts, task plan, enriched context, incidents, severities, Critic verdicts, prose checks, playbooks, approvals, tickets, report).
- **Model:** Ollama running llama3.2 3B locally. It plans the Research agent's tool calls and writes summaries, the report, the handover and chat answers. It never scores, routes or passes a check.

- **Outputs and persistence:** the web console with its exports (PDF report, tickets JSON, Sigma YAML), and the analyst memory file that carries lessons to the next run.

4. The six agents and their tools

Each agent is one LangGraph node: a Python function that reads the shared state and returns its update. Every tool call goes through a tracer, so it appears in the live trace in the console.

#	Agent	What it decides	Tools it uses	Uses the LLM?
1	Planner	Which alerts belong together: groups by host and a 30-minute window into candidate incidents (the task plan)	Grouping logic	No
2	Research	Which lookups each candidate needs. For suspicious candidates the LLM plans the calls; quiet ones get a fixed checklist	`threat_intel.lookup`, `assets.get`, `users.baseline`, `hunt.indicator`, analyst memory	Plans tool calls (validated)
3	Executor	Which candidates are one intrusion across hosts, which kill-chain stages are proven, and the severity score	Correlation engine, cross-host linker, MITRE ATT&CK mapping, rubric	Writes the incident summary (fact-checked)
4	Critic	Whether the work is complete and true; if not, which agent must redo it	Independent score recomputation, lookup audit, rule tester, playbook checker, fact-checker	No
5	Software Development	Response playbook, detection rule, its unit test and Sigma export	Rule generator (brute force, phishing to C2, lateral movement, generic templates), rule tester, pySigma to Splunk and Elastic	No
6	Business Automation	Ticket priority and SLA, notifications, the report and shift handover	Mock ticketing, mock notifier, PDF report writer	Writes report and handover (fact-checked)

Research plan validation. The LLM returns JSON naming tools and arguments. Python rejects any tool that does not exist and any argument (indicator, host or user) that is not in that candidate's own alerts. The model cannot make the system look up something it invented.

Cross-host linking. Two candidates on different hosts merge into one incident when an alert on one comes from the other host's IP address, for the same user, within 60 minutes. This is how credential dumping on a workstation is joined to lateral movement onto the domain controller.

Ticket SLAs. Critical incidents get 30 minutes, high 60 minutes and medium 240 minutes; low-severity incidents are closed with a logged justification.

Playbook rule. Every playbook must contain preserve evidence, contain, eradicate and recover steps, with evidence preserved before the host is isolated. Critic checkpoint 2 enforces this order.

5. Orchestration: the LangGraph state graph

The orchestrator is a LangGraph `StateGraph` with nine nodes. Plain edges fix the order; three conditional edges make the decisions: where the Critic sends work, whether anything needs a playbook, and whether checkpoint 2 passes.

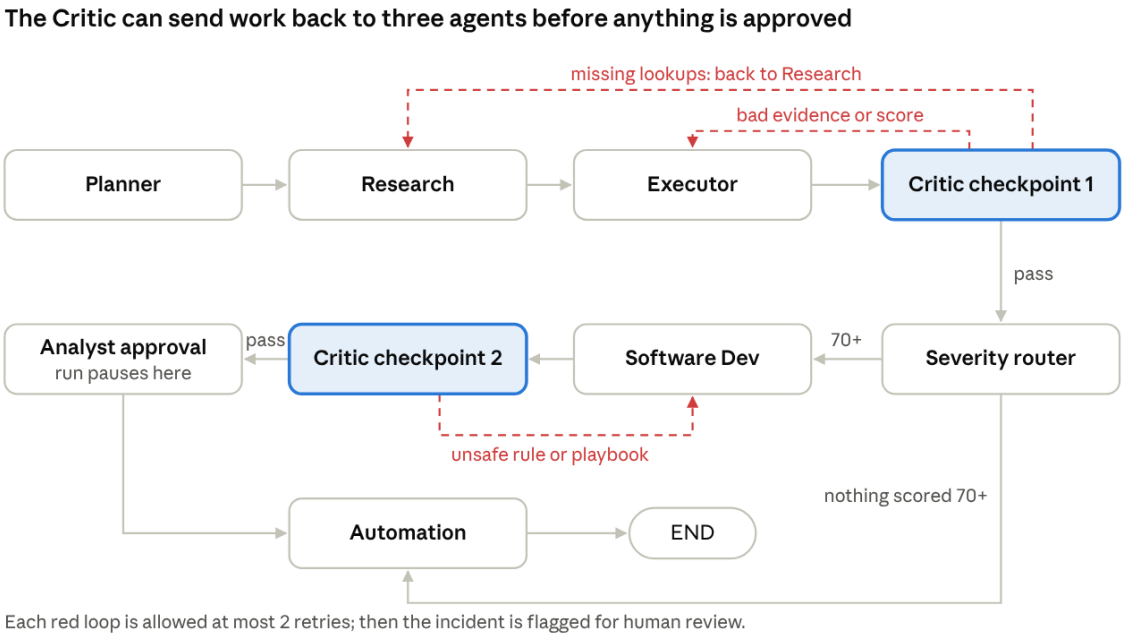


Figure 2. The orchestration graph. Red dashed edges are Critic retries.

Decision point	Function in graph.py	Possible next nodes
After Critic checkpoint 1	`after_cp1`	Research (lookups missing), Executor (evidence or score wrong), Router (pass)
After the router	`after_router`	Software Development (any incident scored 70+), Automation (none)
After Critic checkpoint 2	`after_cp2`	Software Development (retry), Analyst approval (pass)

Incidents below 70 still reach Automation: those under 40 are auto-closed with a logged reason, and 40 to 69 go to the analyst queue with a ticket and an investigation checklist. The web console draws this same graph live, lighting each node as it runs.

6. Severity scoring and decision logic

Severity is a transparent 0 to 100 sum of four parts, computed in `core/rubric.py`. The Executor computes it. The Critic recomputes it from the cited evidence alone, without seeing the Executor's number, and the two must match.

```
severity = min(100, max(0, stage + asset + intel + behaviour))
```

Component	Points
Kill-chain stages proven by the alerts	15 per distinct stage, at most 45
Asset criticality (most critical host involved)	low 0, medium 10, high 20, crown jewel 25
Threat intel verdict (worst indicator)	malicious 20, suspicious 10, unknown 0, benign minus 10
Behaviour against the user's baseline	anomalous 10, normal 0

The router then sends each incident one of three ways: below 40 it is auto-closed with a logged reason, 40 to 69 it goes to the analyst queue, and 70 or more gets a playbook, a tested detection rule and an approval request. Ticket priority follows the same bands (critical at 90 and above).

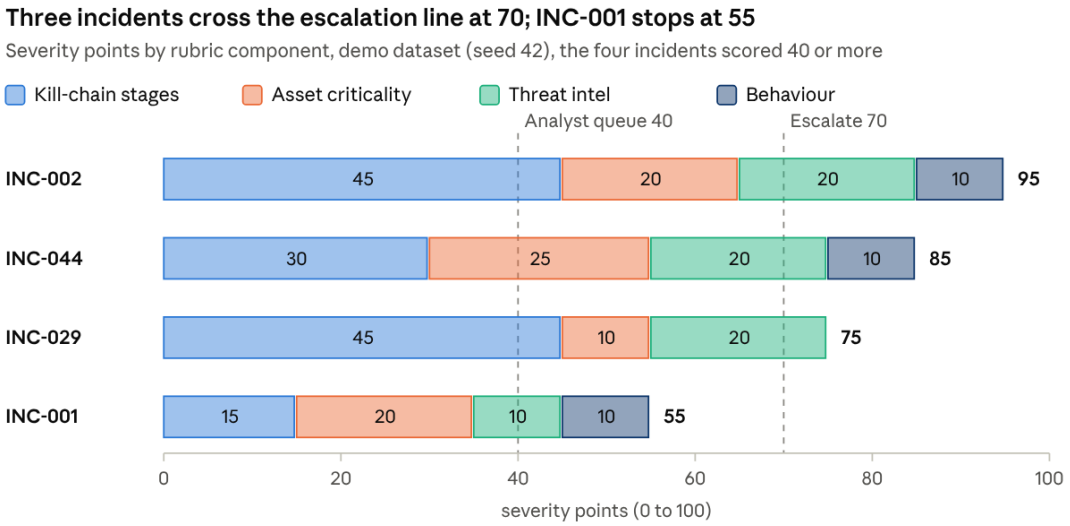


Figure 3. Score breakdown of the incidents that need a person. Every point can be traced to the alerts the claim cites.

INC-001 is the off-hours exfiltration chain. It has one proven stage, so it reaches the analyst queue but not escalation, which matches its expected medium band. Because every point comes from a named component, an analyst can see exactly why an incident was ranked where it was.

7. Validation: the Critic

The Critic never trusts another agent's output: it re-derives what it can from the raw alerts and compares. A failed check sends the work back to the agent that produced it, with the reason attached. Each checkpoint allows at most 2 retries; after that the incident is flagged for human review instead of looping forever.

Check	Checkpoint	What it verifies	On failure
Lookup completeness	CP1	Every indicator, host and user in the candidate's alerts was looked up	Back to Research, which runs only the missing lookups
Evidence	CP1	Every claim cites alert IDs that exist and actually support the claimed kill-chain stage	Back to the Executor

Score	CP1	Severity recomputed from the cited evidence alone, without reading the Executor's number first, matches	Back to the Executor
Detection rule	CP2	The rule fires on the attack's own alerts and stays silent on benign logs (its unit test)	Back to Software Development
Playbook safety	CP2	All four phases present; evidence preserved before containment	Back to Software Development
Prose fact-check	Every LLM text	Every incident ID, alert ID, IP address, host, hash, domain and number in the text appears in the facts the LLM was given	LLM rewrites with the rejection reason (max 2), then a plain factual template

Fault injection. To show the Critic working, faults can be injected on purpose. There are seven types: `skip_lookup` (Research); `drop_evidence`, `inflate_score`, `fake_stage` and `hallucinate` (Executor); and `broad_rule` and `unsafe_playbook` (Software Development). The console offers two modes. “Critic loops” makes each checkpoint reject once and then pass. “Persistent” repeats the fault on every attempt, so retries run out and a person takes over.

Why the fact-check matters. A 3B local model writes fluent but sometimes wrong text. In testing it occasionally invented an IP address or changed a score. The fact-check uses regular expressions to pull every checkable token from the text and compares each one with the facts. It cannot judge style, but it stops the report from stating anything that is not in the data.

8. Human in the loop and analyst memory

Containment is never automatic. After Critic checkpoint 2 the graph calls LangGraph's `interrupt()`, which saves the state with an in-memory checkpointer and pauses the run. The console shows each escalated incident with its proposed containment steps. The analyst chooses one of three options, and the run resumes with `Command(resume=decisions)`:

Analyst decision	**Effect on this run**	**Effect on analyst memory**
Approve	Containment authorised; ticket and report record it	The incident's malicious indicators are block-listed
Hold	Containment put on hold	Nothing learned
False positive	Containment cancelled	Indicators allow-listed; the user's baseline widened to include the host and hours seen

Analysts can also mark any incident “confirmed” or “false positive” from the incident detail view after a run.

How memory changes the next run. The memory is a small JSON file. The Research agent's lookups read it before the mock threat feed and the baselines, so an allow-listed indicator comes back as benign (minus 10 points) and a block-listed one as malicious (plus 20). A widened baseline stops the same activity from counting as anomalous. The trace labels these results

“analyst memory”, so it is always clear why a score changed. Memory can be viewed, edited per entry or cleared in the console.

9. Implementation

The repository has 55 files (about 9,700 lines including data), written in Python 3.11 with a plain HTML, CSS and JavaScript front end (no build step). Everything runs on one laptop.

Layer	Technology	Role
Orchestration	LangGraph 1.2	State graph, conditional edges, retry loops, `interrupt()` and checkpointer for the analyst pause
Language model	Ollama, llama3.2 3B, temperature 0	Research planning (JSON mode) and prose; a deterministic mock model for tests and the benchmark
Backend	FastAPI, Uvicorn	Runs the graph in a background thread and streams each finished node to the browser over Server-Sent Events
Front end	HTML, CSS, JavaScript	Live orchestration graph, agent trace, views for incidents, ATT&CK, response, reports, evaluation, benchmark, memory and chat
Detection export	pySigma with Splunk and Elasticsearch backends	Converts each Sigma rule to SPL and Lucene queries
Reports	ReportLab	PDF report and shift handover
Testing	pytest	28 tests that need neither Ollama nor a network

Code layout. `graph.py` wires the graph. `agents/` holds one file per agent. `tools/` holds what agents call: enrichment, correlation, MITRE mapping, rule generation and testing, Sigma export, playbooks, ticketing and Windows log ingestion. `core/` holds the shared state, rubric, LLM clients, tracer, faults, memory, Q&A and exports. `eval/` and `tests/` measure and test it.

Web console. A run streams into a live orchestration graph: each node lights up as its agent works, and packets travel along the edges, including every Critic loop back. Beside it are the agent trace, a Critic feedback panel and live counters. After the run the console offers:

- incident detail with timeline, score breakdown, Critic verdicts and the LLM's research plan;
- a MITRE ATT&CK coverage grid;
- the playbook, the detection rule and its Sigma, Splunk and Elastic versions;
- the report, evaluation against ground truth, the benchmark and analyst memory;
- an “Ask the run” chat.

The chat answers from facts retrieved in Python, has the LLM phrase the answer, and fact-checks it like any other prose.

Fail loudly. The console uses Ollama only. If the model is missing or stops answering mid-run, the run stops with a clear error rather than quietly switching to template text.

10. Data and experimental setup

All data is synthetic or hand-written in a public log format; no real organisation's logs are used. A seeded generator (`scripts/generate_data.py --seed N --alerts M`) hides four attack chains in benign noise. Hosts, users, indicators and timings change with the seed, so the system cannot rely on fixed values.

Demo dataset (seed 42). 153 alerts: 26 belong to attacks and 127 are benign. By source they are 96 authentication, 34 EDR, 16 firewall and 7 DNS alerts. The generator also writes the reference data the tools read: an asset inventory with criticality, a mock threat-intel feed and user baselines (usual hosts and working hours).

Chain	Attack	Alerts	Hosts	Expected severity
A	Brute force, then external login, then a new admin account	11	1 (application server)	High
B	Phishing link, then malware download, then C2 beaconing	8	1 (marketing workstation)	High
C	Off-hours large outbound transfer (exfiltration)	3	1 (finance workstation)	Medium
D	Credential dumping, then lateral movement to a database server and the domain controller	4	3	High

Windows Security and Sysmon sample. 24 events written by hand with real Windows field names (Security 4625, 4624 and 4732, System 7045, Sysmon 10, 15 and 22), which become 23 alerts, as in the public OTRF Security-Datasets. `tools/ingest.py` maps them to the same alert format, so the agents run on it unchanged. Users can also upload their own JSON.

Experiments

1. **Demo evaluation:** one run on the seed 42 dataset scored against `ground_truth.json`.
2. **Benchmark:** 15 datasets (seeds 1 to 5, each at 150, 300 and 600 alerts). Metrics: precision, recall and F1 on attack alerts, where an attack alert counts as found when it lands in an incident scored 40 or more; chains detected in the expected severity band; benign incidents escalated.
3. **Critic ablation:** 42 faults (each of the 7 types 6 times, on a random dataset and target), run once with the Critic enforcing and once with it disabled. Disabled means its checks still run and are recorded, but it never sends work back.

The benchmark and ablation use the deterministic mock model so they are repeatable and need no GPU. Runs with the real Ollama model were checked separately (section 11).

11. Results and evaluation

The system found every attack chain in the demo dataset with the expected severity and escalated no benign activity. It found 57 of 60 chains across 15 generated datasets, and the

Critic stopped all 42 injected faults. Results come from `python eval/run_eval.py` and 28 passing tests.

11.1 Demo dataset (seed 42, 153 alerts)

Chain	Incident	Score	Band (expected)	Attack alerts grouped	Hosts
A: brute force to admin account	INC-002	95	high (high)	11 of 11	1
D: credential dumping to domain controller	INC-044	85	high (high)	4 of 4	3
B: phishing to C2 beaconing	INC-029	75	high (high)	8 of 8	1
C: off-hours exfiltration	INC-001	55	medium (medium)	3 of 3	1

All 4 chains were correlated (4/4). None of the 127 benign alerts was escalated. Alert-level precision, recall and F1 were all 1.0, and all 3 generated detection rules passed their tests.

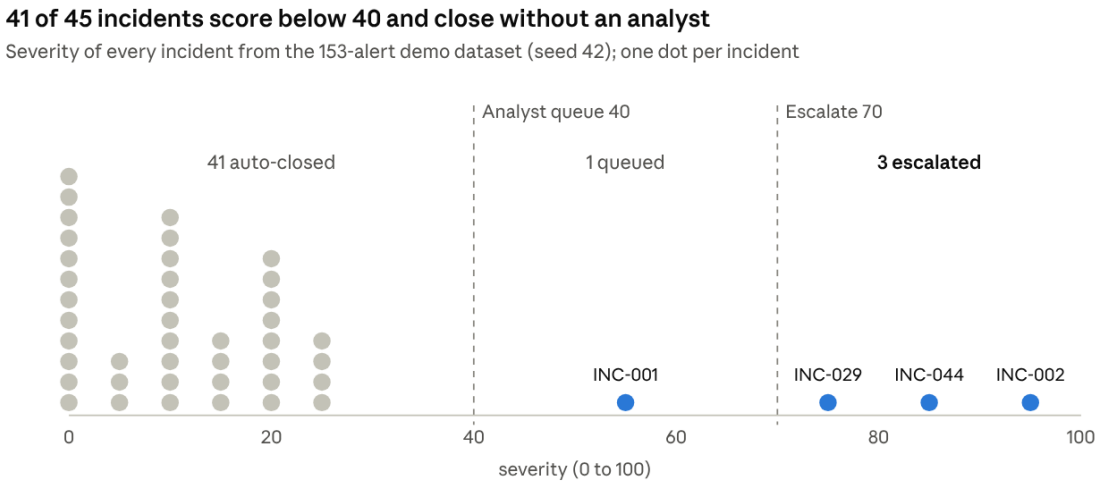


Figure 4. Severity of all 45 incidents. Benign noise sits far below the 40-point line; the four real attacks sit above it.

This is the alert-fatigue result: an analyst looks at 4 incidents instead of 153 alerts, and every closed incident carries a logged justification.

11.2 Benchmark across 15 datasets

Alerts per dataset	Datasets (seeds 1 to 5)	Precision	Recall	F1	Chains detected	Benign incidents escalated
150	5	1.000	1.000	1.000	19 of 20	0
300	5	1.000	1.000	1.000	19 of 20	0
600	5	1.000	1.000	1.000	19 of 20	0

Precision and recall did not change as noise grew fourfold. All three misses are the same case: in seed 1 the phishing chain lands on a low-criticality host and scores 65, so it reaches the analyst queue instead of the high band. Its alerts were still found and queued for a person.

11.3 Critic ablation

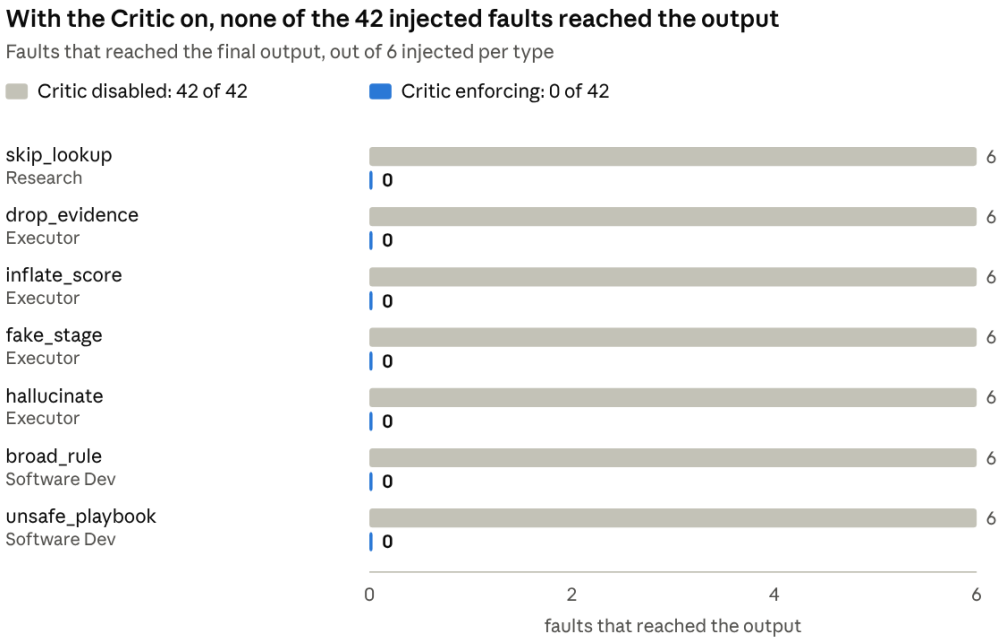


Figure 5. The same 42 faults run twice: once with the Critic enforcing, once with it only observing.

With the Critic enforcing, all 42 faults were caught and corrected by a retry. With it disabled, all 42 shipped: wrong scores, invented kill-chain stages, hallucinated text, rules that fire on benign logs and playbooks that isolate a host before preserving evidence.

11.4 Runs with the real Ollama model

Run (llama3.2 3B on a laptop CPU)	Time	Critic retries	LLM text rewrites	Outcome
Synthetic dataset with "Critic loops" faults and analyst approval	97 s	1 at CP1, 1 at CP2	3	4 of 4 chains, same scores as the table above
Windows Security and Sysmon sample, no faults	54 s	1 at CP1	0	2 escalated, 1 queued, 2 auto-closed

In the Windows run no fault was injected: the LLM's own research plan missed a lookup, and the Critic caught it and sent it back. This is the behaviour the Critic exists for, happening on its own.

12. Demo walkthrough

A five-minute demo shows every required behaviour in one run with real Ollama:

1. **Set up** (20 s). Select the synthetic dataset, turn fault injection to "Critic loops" and keep analyst approval on.
2. **Planning and tool use** (60 s). Press Run triage. The Planner groups the 153 alerts into candidate incidents (45 after cross-host linking). Research shows the LLM's plan for each suspicious candidate in the trace: which lookups, and why.

3. **Validation** (60 s). Critic checkpoint 1 rejects the Research work for a missing lookup; the graph edge back to Research lights up and only the missing lookup reruns. Checkpoint 2 then rejects an unsafe rule or playbook, and Software Development regenerates it.
4. **Human decision** (40 s). The run pauses. Approve containment for the top incident (INC-002, severity 95); the run resumes.
5. **Outputs** (60 s). Show the incident detail and score breakdown, the ATT&CK grid, the playbook, and the Sigma rule converted to Splunk.
6. **Grounded chat and evidence** (40 s). Ask “Which incident is most severe and why?” Then show the Eval tab (4/4 chains) and the Benchmark tab's ablation table.

In a recorded check this full run took 97 seconds on a laptop CPU. The Critic sent work back once at each checkpoint, the LLM's prose was rewritten 3 times after the fact-check, and all 4 chains were found with the expected severity bands. Start Ollama before presenting so the model is already loaded.

13. Limitations and future work

The strong scores come from scripted attacks that the correlation rules were written to recognise. They show the pipeline works end to end, not that it would catch a new attack technique.

Limitations

- **Synthetic data.** The attacks follow four scripted patterns, and the Windows sample is small and hand-written.
- **Fixed rules.** Correlation rules, thresholds and rubric weights are set by hand. One benchmark miss shows the edge: phishing on a low-criticality host scores 65, just under the high band.
- **Mock integrations.** Threat intel, the asset inventory, ticketing and notifications are local files and mocks.
- **Small model.** A 3B model writes plain, sometimes clumsy prose. The fact-check stops it inventing IDs, IPs, hosts or numbers, but not from being awkward.
- **Not deployable as is.** The console has no login, keeps runs in memory and needs Ollama on the same machine.

Future work

- Connect to a real SIEM (Splunk or Elastic) and a real threat-intel source such as MISP or AbuseIPDB.
- Evaluate on public datasets such as the full OTRF Security-Datasets or Mordor scenarios.
- Learn rubric weights from analyst verdicts in memory instead of fixing them by hand.
- Add authentication, persistent run storage and role-based approval for real deployment.

- Try a larger local model for better prose and planning, keeping the same Critic checks.

14. Conclusion

The project shows that a team of agents can triage a SOC alert batch end to end on a laptop, with no paid API, and that it can be trusted when the language model is kept away from decisions. The LLM plans investigations and writes text; Python scores, routes and checks. The Critic is what makes the output reliable. In the ablation it caught and corrected all 42 injected faults, and every one of them reached the final output when it was switched off.

The main lesson is that agentic systems need validation built into the graph, not added afterwards. Each claim must cite evidence, each score must be recomputed independently, and each sentence must be checked against the facts. A person must still approve any action that changes a system.

15. References

1. LangGraph, LangChain: langchain-ai.github.io/langgraph
2. Ollama: ollama.com; Llama 3.2 3B model: ollama.com/library/llama3.2
3. MITRE ATT&CK Enterprise matrix: attack.mitre.org
4. Sigma detection rule format: sigmahq.io; pySigma: github.com/SigmaHQ/pySigma
5. OTRF Security-Datasets (Windows Security and Sysmon field names): github.com/OTRF/Security-Datasets
6. FastAPI: fastapi.tiangolo.com
7. Project source code: github.com/da-ksh-esh/SOCAgent

Appendix: how to run

Requires Python 3.11 and Ollama.

```
pip install -r requirements.txt
ollama pull llama3.2
python scripts/generate_data.py
python server.py
```

Then open `http://localhost:8000`. To reproduce the evaluation, run `python eval/run_eval.py`; the tests run with `pytest`. Optional settings: `OLLAMA_HOST`, `SOC_HOST` and `SOC_PORT`.